

Organoid Agent — Project Notes

Diya Sreedhar (Arlotta Lab MVP)

v0.2 — answers to Q1 and Q2

Contents

Vision	1
Question 1 — what model innovations / tools / datasets?	2
1a. Architecture pattern (what the agent looks like)	2
1b. Tools to add (organoid-specific)	2
1c. Datasets to include	3
1d. What “model innovation” actually means here	4
Question 2 — what benchmarks?	5
2a. Benchmark tasks (HNOCA-grounded)	5
2b. Ablation studies (per TxAgent / Medea methodology)	6
2c. T7 design — the agent-vs-LLM eval	6
2d. What “success” looks like for the Arlotta presentation	6
Where we are right now (the journey to date)	7
Initial Plan (what was already in the repo)	7
Modified Plan — stream just what we need	7
New Plan — cellflow / Medea-style agent layer	8
Roadmap to Q1/Q2 answers fully implemented	8
Appendix A — Understanding HNOCA	9
Appendix B — Useful Commands	9
Appendix C — How to reproduce from scratch	10
Appendix D — Open questions / things to bring up with the lab	11
Appendix E — misc surprises from real data	11

Vision

Build an **LLM-powered agent that orchestrates tools** to do science on brain organoids:

- *calls to pre-trained models* (e.g. CellFlow, scGPT, Geneformer, scFoundation, UCE),
- *API calls to PubMed / OpenAlex / OpenScholar* to retrieve organoid papers,
- *code generation* to query HNOCA and other brain-organoid atlases,
- *code generation* to run downstream analyses on those queries.

The end goal is an **AI scientist for brain organoids** that makes predictions about private Arlotta-Lab datasets, which the lab then validates at the bench. Before we get there, we need to demonstrate a competent agent that beats current frontier models on a **public benchmark** built from HNOCA. That public benchmark is what this week’s questions are about.

Two reference architectures we’re explicitly modelling on:

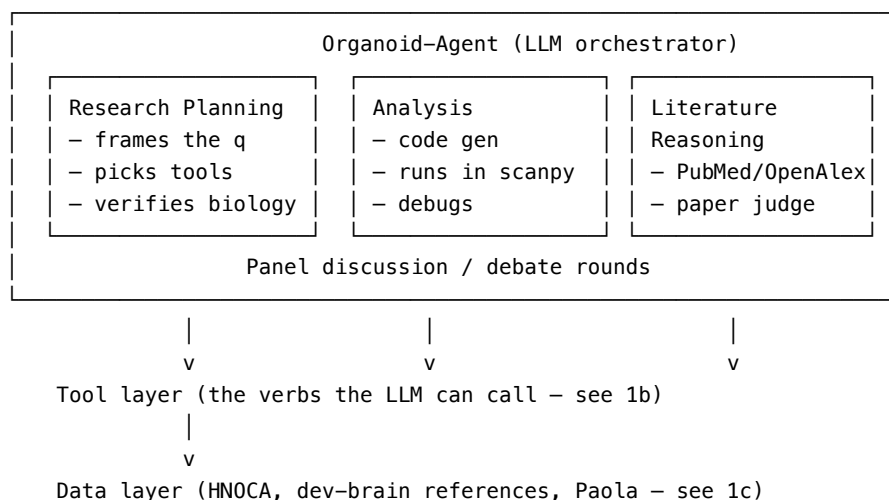
- **Medea** ([mims-harvard/medea](https://mims-harvard.github.io/medea), [bioRxiv 2026](https://doi.org/10.1101/2026)) — an omics AI agent for therapeutic discovery. **Three collaborating modules** (Research Planning, Analysis = code-gen, Literature Reasoning), 17 tools, debate rounds across a panel of LLMs. Built on AgentLite; uses ToolUniverse for tool management. Evaluated on three tasks: TargetID, Synthetic Lethality, Immune Therapy Response.
- **TxAgent** ([arXiv:2503.10970](https://arxiv.org/abs/2503.10970)) — a precision-medicine agent with **211 tools**, 92.1% on open-ended drug reasoning, 5 new benchmarks (DrugPC, BrandPC, GenericPC, TreatmentPC, DescriptionPC). Useful reference for tool-ablation methodology.

Reference review for what AI is doing in organoids today: **Ramesan et al. 2026**, “Next-generation discovery: empowering organoid research with machine learning, AI, and mathematical modeling,” *Trends in Biotechnology*.

Question 1 — what model innovations / tools / datasets?

1a. Architecture pattern (what the agent looks like)

Mirror Medea’s three-module design, specialised for organoid biology:



Key innovation vs Medea: **the analysis module needs to know about single-cell / scRNA-seq specifically**. Medea’s CodeGenerator does generic omics; ours needs to be primed with scanpy / anndata / scvi-tools idioms, HNOCA’s obs schema, and the failure modes of organoid data (batch effects, protocol confounders, label-space drift across studies).

1b. Tools to add (organoid-specific)

Grouped by what they do. Names in `code font` are concrete things we can implement; **bold** are *high-priority for v1* (i.e. what we should ship for the Arlotta presentation).

Atlas query / retrieval

- **query_hnoca(region, protocol, age_min, age_max, label_col)** — what cells in the atlas match these criteria? Returns composition, marker-gene means, kNN neighbours. *Already implemented* in `agent/hnoca_model.py` as four sub-tools.
- `map_query_to_atlas(adata)` — wrapper around [HNOCA-tools map_query](#); projects a query AnnData (Paola’s organoids) into HNOCA’s scANVI latent and returns nearest-atlas-cells + presumptive labels.
- `compare_to_primary(cells)` — score each query cell against HNOCA’s curated primary developing-brain reference. **The fidelity signal** the HNOCA paper emphasises.

Foundation-model embeddings (call as tools, like Medea’s `transcriptformer_embedding`)

- `scgpt_embed(adata)` — `scGPT` (Cui et al. *Nat. Methods* 2024). Transformer pretrained on 33M cells.
- `geneformer_embed(adata)` — `Geneformer` (Theodoris et al. *Nature* 2023). Rank-based gene tokens; strong for zero-shot tasks.
- `scfoundation_embed(adata)` — `scFoundation` (Hao et al. *Nat. Methods* 2024). Large pretrained.
- `uce_embed(adata)` — `Universal Cell Embedding` (Rosen et al. 2023). Pretrained on 36M cells across 1,000+ tissues; good for cross-species.
- `transcriptformer_embed(adata)` — `Transcriptformer`. Same wrapper Medea ships.

Generative perturbation / trajectory predictors

- `cellflow_predict(condition)` — `CellFlow` (Klein, Fleck et al. 2025); flow-matching + OT for predicting how a cell state distribution shifts under a perturbation. *The headline ML tool* the lab is asking us to use.
- `moscot_pseudotime(adata)` — `moscot` neural OT, the pseudotime backbone HNOCA itself uses.
- `scgen_predict(adata)` — older but well-validated perturbation predictor.

Literature / hypothesis lookup (Medea-style)

- `pubmed_search(query)` — direct port of Medea’s `pubmed_search.py`.
- `openalex_search(query)` — OpenAlex API for open-access metadata (Medea ships `open_alex.py`).
- `openscholar_reason(question)` — Medea’s `OpenScholarReasoning` action; use it to filter and synthesize papers per question.
- `paper_judge(paper, question)` — relevance scoring, again from Medea.

Code generation + execution (Medea-style)

- `scanpy_codegen(task) → execute(code)` — generate scanpy/anndata Python for ad-hoc queries (e.g. “*plot a UMAP of Velasco cells coloured by NEUROD6*”), execute in a sandboxed kernel, capture stdout + figures. Medea’s `CodeGenerator` + `AnalysisExecution` + `CodeDebug` is the template.

Bench-relevance / discovery (where the Arlotta connection lives)

- `find_novel_cells(query_adata, atlas)` — score cells by latent-space distance to nearest HNOCA neighbours; flag the top-N% as candidates for *novel* states.
- `trajectory_deviation(query_adata, atlas, protocol)` — does the query follow the closest atlas trajectory under the same protocol? Quantify with KL-divergence on composition.
- `marker_gene_proposer(unmatched_cluster)` — given an unmatched cluster, return its top DE genes vs. nearest atlas type → wet-lab targets.

Tool-management framework

Use `ToolUniverse` — the same registry Medea uses, with a JSON-schema config per tool. Free benefit: Medea’s existing tools (PubMed search, code gen, paper judge) plug in directly.

1c. Datasets to include

Tier 1 — required for v1 benchmark

Dataset	Size	Where	Why
HNOCA cleaned atlas	1.77M cells × 36k genes	Zenodo 14161275 → <code>hnoca_cleanedmeta.h5ad</code> (17.5 GB)	The benchmark substrate. 36 datasets, 26 protocols, days 7-450. We stream subsets (scripts/download_hnoca_subset.py) instead of full download.

Dataset	Size	Where	Why
Braun et al. dev-brain reference	1.6M fetal brain cells	GSE166854 / Linnarsson lab	HNOCA’s <i>primary reference</i> for fidelity scoring. T3 needs this.
CellxGene HNOCA mirror	same as Zenodo	collection de379e5f-52d0-498c-9801-0f850823c847	Backup access + standard schema.

Tier 2 — orthogonal validation / scope expansion

Dataset	Where	Why
iNeurons morphogen screen (CellFlow training data)	theislab/cellflow-datasets (gated)	Real perturbation pairs — gives us a T4 (perturbation response) signal CellFlow was trained for.
NOHA — Neural Organoid Hormonal Atlas	CellxGene 2bebc4ae-69a2-4001-9d2f-091ec9b4020c	Hormonal-axis perturbations on neural organoids; complementary to HNOCA’s developmental axis.
Cerebellar organoid atlas	CellxGene 0dd101f7-9829-44b3-a323-18b113eabeb4	Region not well-covered by HNOCA.
VPA-treated dorsal forebrain organoids	CellxGene c2879de0-affc-496b-8e2b-f57ed9ec3c34	Drug-treated organoids — directly relevant to “predict response to X” tasks.
HNOCA disease atlas	Zenodo 14161275 → disease_atlas.h5ad (2 GB)	Disease-condition organoids; useful for T4.

Tier 3 — long-tail / reference knowledge

- **OpenTargets** (gene-disease links; Medea’s `load_disease_targets`)
- **DepMap** (cell-line viability for validation; Medea’s `compute_depmap_correlations`)
- **PINNACLE protein embeddings** (cell-type-contextual PPI; Medea’s `load_pinnacle_ppi`)
- **Human Protein Atlas** (tissue-resolved expression)
- **HumanBase** (tissue-specific co-expression / PPI)
- **WikiPathways / Reactome** (pathway context for genes the agent surfaces)

Tier 4 — private (deferred)

- **Arlotta-Lab organoid scRNA-seq** — the final eval target. Not used for benchmark development; the agent should never see it during training/dev.

1d. What “model innovation” actually means here

Three innovations that distinguish a useful organoid agent from “GPT-4o + retrieval”:

1. **Tool-grounded numeric claims.** The LLM never produces a composition fraction, an expression value, or a “% novel cells” from its own weights; every quantitative claim is the literal return value of a tool call. This is the rule in Medea, in our `agent/hnoca_model.py`, and the non-negotiable for credibility with Paola.
2. **Foundation-model-as-tool, not foundation-model-as-replacement.** The agent calls Geneformer/scGPT/CellFlow as backends, picks between them per question, and reports which one it used. Lets us ablate the tool set and quantify what each backbone contributes (the TxAgent paper template for tool ablations).
3. **Atlas-grounded generalisation.** For Paola’s data, generalisation is measured against HNOCA cells, not against a held-out test split of her data. *Novel cell ~no good neighbour in HNOCA; protocol-deviation ~ trajectory mismatch with the closest atlas protocol.*

Question 2 — what benchmarks?

For each task we list: input, target, metric, baseline, what would constitute success on the Arlotta presentation. The lab spreadsheet ([here](#)) defines the formal task list — the table below is our concrete instantiation of those rows against HNOCA.

2a. Benchmark tasks (HNOCA-grounded)

ID	Question	Input → Target	Primary metric	Baselines
T1	Next-timepoint pseudobulk within protocol	pseudobulk(t) → pseudobulk(t+1) within protocol	Pearson, MSE, sign-acc-lineage	identity (x_t), pop-mean, ridge, FM-conditioned ridge
T2	Out-of-protocol cell-type generalisation	cells from {A,B,C} → labels on cells from D	macro-F1, accuracy, per-class F1	logreg, kNN, scGPT / Geneformer head
T3	Primary-reference fidelity	organoid cell → similarity-to-primary	Spearman vs. published score, AUROC on “matched” cells	random, raw-correlation, HNOCA-tools score
T4	Cross-protocol composition shift	Velasco @ age T → Lancaster @ age T	KL on composition, MMD on embeddings	identity (same protocol), CellFlow, FM-conditioned regressor
T5	Real-time pseudotime	cell expression → organoid_age_days	Spearman, Kendall-tau, within-protocol Spearman	PC1, diffusion-pseudotime, moscot, scGPT embed + ridge
T6	Novel-cell detection (atlas mapping)	query cells (Paola or masked HNOCA) → “in atlas” / “novel”	AUROC (held-out HNOCA-tools labels vs. random injections)	random, kNN-distance, HNOCA-tools
T7	Atlas QA (LLM-graded)	NL question → answer grounded in tool calls	accuracy, hallucination rate, tool-call F1	map_query frontier LLM (no tools), retrieval-only LLM, our agent

T1, T2, T5 are already implemented

- T1 in `src/tasks/t1_next_timepoint.py` (group_key="protocol" for HNOCA)
- T2 in `src/tasks/t2_celltype_oop.py`
- T5 in `src/tasks/t5_pseudotime.py`

The numbers from `make plots` on the real Dorsal-telencephalon subset:

Task	Model	Metric	Real HNOCA
T1	identity	Pearson	0.48
T1	pop_mean	Pearson	0.46
T1	linear (ridge)	Pearson	0.19
T1	pop_mean	MSE	0.26
T2	logreg	macro-F1	0.16
T2	knn	macro-F1	0.10
T5	PC1	Spearman	0.25

Headroom is huge — a foundation-model embedding head should beat these by a wide margin.

T3, T4, T6, T7 are the next chunk of work

- **T3** needs the primary-similarity column added to the streamed subset (it’s an obs column in the full atlas; we didn’t pull it the first time). One-line fix in `scripts/download_hnoca_subset.py`.
- **T4** needs (i) at least two protocols with matched ages and $\geq$ dense cell counts — Velasco and Lancaster are the cleanest pair in our subset; (ii) a CellFlow checkpoint we can call. The [theis-lab/cellflow_reproducibility](#) repo has trained models.
- **T6** is the closest analogue to the Arlotta use case. Construction: hold out one HNOCA *region* (e.g. cerebellum), inject those cells into the otherwise-trained query, see whether the agent correctly flags them as novel. AUROC against the held-out region label is the metric.
- **T7** is the **agent-vs-LLM** benchmark — see 2c.

2b. Ablation studies (per TxAgent / Medea methodology)

Following the [TxAgent ablation pattern](#):

1. **Tool-by-tool removal.** Run the full agent on T1–T7, then re-run with each tool individually disabled. Δ accuracy per tool = its marginal contribution.
2. **Tool-substitution.** Swap `scgpt_embed` $\leftrightarrow$ `geneformer_embed` $\leftrightarrow$ `uce_embed` $\leftrightarrow$ `transcript-former_embed`. Same agent, same prompts, different foundation-model backbone $\rightarrow$ which one ports best to organoid biology?
3. **Oracle baselines.** Give the agent gold-standard tool outputs (e.g. the correct cell-type labels handed in) — measures *reasoning headroom* independent of tool error.
4. **Frontier-LLM baseline.** Same questions to GPT-5 / Claude / Gemini *without any tools*, only their parametric knowledge. Quantifies the value of grounded tool calls vs. text-only generation.
5. **Retrieval-only baseline.** PubMed + OpenAlex retrieval, no foundation-model embeddings, no code-gen — what does literature *alone* get us?

2c. T7 design — the agent-vs-LLM eval

This is the headline result we’d want for the Arlotta presentation.

- **Build set.** Generate ~ 200 NL questions whose ground-truth is a measurable quantity in HNOCA (e.g. “*What fraction of Velasco day-100 cells are pyramidal neurons?*”, “*At what age does NEUROD6 peak in cortical organoids?*”, “*Is BCL11B expressed in day-30 organoids?*”).
- **Ground truth.** Computed once from HNOCA by hand-written tool calls; cached.
- **Judge.** LLM-graded with a Medea-style PaperJudge-equivalent — binary “matches ground truth $\pm$ tolerance”.
- **Compared systems.**
 1. Our agent with full tool set
 2. Our agent with tools, but no foundation-model backbones
 3. Frontier LLM (no tools)
 4. Frontier LLM + naive RAG over HNOCA paper PDFs only
- **Headline metrics.** Accuracy, hallucination rate (% claims not backed by a tool return), tool-call F1 against a gold trajectory.

2d. What “success” looks like for the Arlotta presentation

- Our agent $\geq +15$ **accuracy points** over the best frontier-LLM-no-tools baseline on T7.
- T4 (cross-protocol composition shift): our CellFlow-as-tool prediction $\geq$ identity baseline by KL on at least 3 protocol pairs.
- T6 (novel-cell detection): AUROC ≥ 0.85 on a held-out-region reconstruction.
- A clean tool-ablation plot that says “Geneformer alone gets X; +CellFlow $\rightarrow$ X+ Δ ; +PubMed retrieval $\rightarrow$ X+ Δ ”.
- A 1-paragraph “what we would do with Arlotta organoids” pitch with concrete tool calls.

Where we are right now (the journey to date)

This is the project log — what was already in the repo, the four pivots, and the current state. Useful context for the questions above, but not the answer itself.

Initial Plan (what was already in the repo)

- a) **Data.** Pull cleaned HNOCA from Zenodo, subset to a region.
- b) **Experiment.** Five candidate tasks (T1–T5) with baselines.
- c) **Figures.** Leaderboard per task + pred-vs-true scatter.

```
make_synthetic_hnoca() → Task.prepare → Model.fit/predict → metrics
  (fake counts)           (T1/T2/T5)      (identity, ridge, (Pearson,
                                         logreg, KNN, PC1)    F1, ρ)
                        |
                        v
                    plots/*.png + results_long.csv
```

Pivot 1. The plots advertised as HNOCA were computed on `make_synthetic_hnoca()` — a generator that mimics the obs schema but invents counts. README claimed real numbers; nothing in the repo ever touched Zenodo. The “0.995 Pearson on T1 Linear” was a self-congratulatory smoke test, not a benchmark.

Pivot 2. Real HNOCA is 17.5 GB cleaned / 49 GB full. The `disease_atlas.h5ad` (2 GB) is different scope. HNOCA-tools doesn’t ship an example. Needed a different access pattern.

Modified Plan — stream just what we need

Open the remote `h5ad` with `fsspec+h5py`, read only `/obs`, pick rows, read only those CSR rows. Bandwidth: **~80 MB instead of 17.5 GB.**

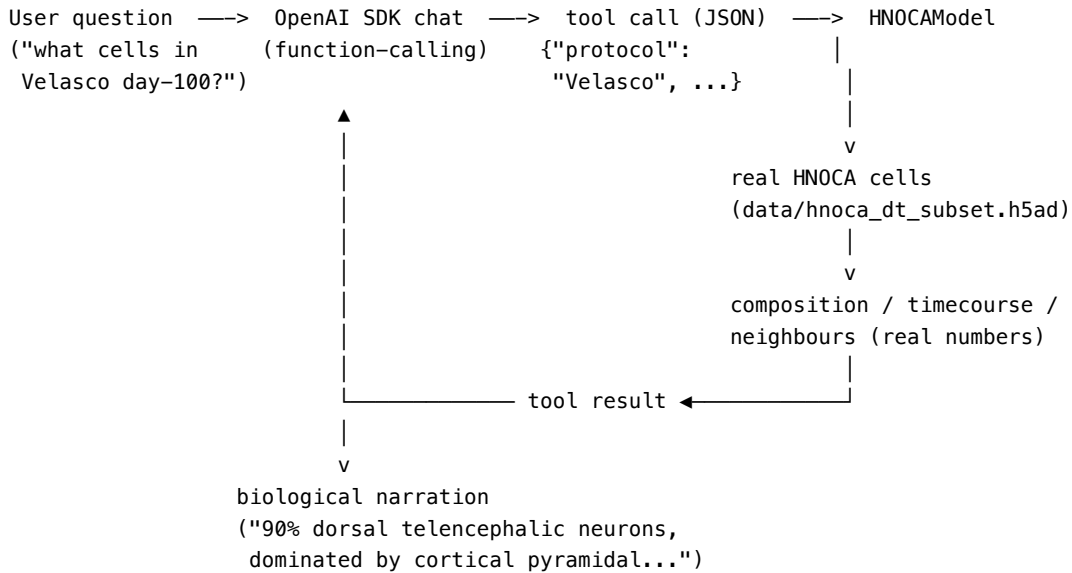
```
Zenodo .h5ad —fsspec HTTP range reads—> h5py.File(remote, "r")
(17.5 GB
cleanedmeta)
|
|—> read /obs columns (~50 MB)
|—> pick row_idx (region + timepoint)
|—> read /X CSR rows in bounded slabs
|
v
data/hnoca_dt_subset.h5ad (~80 MB)
|
├── v ─── explore_hnoca.py ─── v ─── 01_explore.py
      (sectioned printout)          (plots + leaderboards)
```

Pivot 3. Wired real data in. T1 broke — in HNOCA each `bio_sample` is one snapshot. **Only 1/206 organoids had ≥ 2 timepoints.** That’s a biology-shaped fact, not a bug: HNOCA pools across studies, so time signal lives *between* organoids of different ages, not *within*. Switched T1 to group by `protocol` → 13/16 protocols usable.

Real-data leaderboard (honest after the fix) — table reproduced in 2a.

Pivot 4. Re-read the repo name: `organoid-agent`. No agent. No `agent/`, no LLM, no tool-calling, no OpenAI SDK. The repo was a benchmark scaffold pretending to be an agent. Looked at [mims-harvard/cellflow](#) and [mims-harvard/medea](#) for the reference shape.

New Plan — cellflow / Medea-style agent layer



Current state of the agent layer:

- 4 tools (query_composition, gene_expression_timecourse, predict_composition_at_age, find_similar_cells) — all answer from real cells.
- OpenAI SDK chat loop with function-calling, system prompt built from describe_inputs() so the LLM can only pick from real protocols / ages / cell types.
- make agent / make demo / make plots shortcuts.
- README rewrite in cellflow's voice.

Roadmap to Q1/Q2 answers fully implemented

Step	Status	Notes
Stream real HNOCA subset	done	data/hnoca_dt_subset.h5ad, ~80 MB
Regenerate plots from real cells	done	plots/*.png, honest numbers
Agent layer with 4 tools	done	agent/hnoca_model.py
Mirror cellflow's repo shape	done	Makefile, .env.example, demo/
Add Medea-style 3-module structure	next	research-planning, analysis, lit
Plug Geneformer or scGPT as a tool	next	first foundation-model backbone
Pull sim_to_primary for T3	next	one-line addition to downloader
T4 cross-protocol via CellFlow	next	needs CellFlow checkpoint
T6 novel-cell detection bench	next	held-out-region construction
T7 agent-vs-LLM QA bench	next	200 NL questions + judge
Tool ablation study	next	per TxAgent pattern
Project Paola's organoids	Phase 2	wait until Arlotta presentation lands

Appendix A — Understanding HNOCA

Headline. Integrated transcriptomic atlas of human neural organoids: ~1.7M cells, 36 datasets, 26 differentiation protocols, organoid ages day 7 → 450. Mapped to a curated primary developing-brain reference for fidelity scoring. Built with scanpy 1.9.3, integrated with scVI / scANVI, time-aware pseudotime via moscot OT.

Files on Zenodo 14161275:

File	Size	What it is
hnoca_cleanedmeta.h5ad	17.5 GB	Cleaned atlas, what we stream from
hnoca_extended.h5ad	18.8 GB	Cleaned + extra embeddings
disease_atlas.h5ad	2.0 GB	Disease sub-atlas; different scope

Real obs schema (selected):

annot_region_rev2	Dorsal telencephalon / Ventral telencephalon / Medulla / ...
annot_level_2	Dorsal Telencephalic Neuron / NPC / IP / Astrocyte / OPC / ...
annot_level_3_rev2	Finer subdivision under level_2
organoid_age_days	float, days post-induction (7 – 450)
assay_differentiation	Full protocol citation (e.g. "Velasco, 2019 (doi: ...)")
bio_sample	Organoid ID (~one organoid at one timepoint)
batch	Sequencing batch
publication	First-author short
cell_type	cellxgene-harmonised label

X is sparse CSR: /X/data 3.92 G float32, /X/indices 3.92 G int64, /X/indptr 1.77 M int64. Gzip-compressed, chunk size 59,810. Streaming subsets works because HDF5 is random-access by chunk.

Region distribution (full atlas):

Dorsal telencephalon	757,626 cells	← what we subset to
Unspecific	369,569	
Ventral telencephalon	158,506	
Medulla	144,802	
Cerebellum	99,577	
Thalamus	73,377	
Pons	54,739	

Our Dorsal-telencephalon subset (4,430 cells, 16 protocols, days 15–300): top cell types in 2a above; annot_level_2: 68.6% Dorsal Telencephalic Neuron, 29.8% NPC, 1.6% IP.

Appendix B — Useful Commands

Conda env:

```
conda create -n organoid-agent python=3.11 -y
conda activate organoid-agent
pip install -r requirements.txt
```

Streaming an h5ad over HTTPS (fsspec + h5py):

```
import fsspec, h5py, aiohttp
url = "https://zenodo.org/api/records/14161275/files/hnoca_cleanedmeta.h5ad/content"
timeout = aiohttp.ClientTimeout(total=600, sock_read=300, sock_connect=60)
```

```
fs = fsspec.filesystem("https", client_kwargs={"timeout": timeout})
f = fs.open(url, mode="rb", block_size=8 * 1024 * 1024)
hf = h5py.File(f, "r")
list(hf["obs"].keys()) # all obs columns
```

AnnData introspection:

Command	What it does
<code>adata</code>	Shape + obs/var/uns/obsm summary
<code>adata.obs.columns.tolist()</code>	All cell-level metadata fields
<code>adata.obs["region"].value_counts()</code>	Categorical histogram
<code>adata.obs.groupby("protocol", observed=True)["age_days"].nunique()</code>	Time-coverage per protocol (revealed Pivot 3)
<code>adata.X[0, :10].toarray()</code>	Peek at first cell's first 10 genes

GitHub CLI:

```
gh api repos/OWNER/REPO/contributors --jq '.[0] | "\(.login)\t\(.contributions)'"
gh api repos/OWNER/REPO/stats/contributors # async recompute trigger
```

Makefile shortcuts:

```
make data # stream ~80 MB HNOCA subset
make explore # printed summary
make plots # regenerate plots/
make agent # interactive REPL (needs OPENAI_API_KEY)
make demo # one-shot agent question
make test # pytest sanity
```

Appendix C — How to reproduce from scratch

1. Clone.

```
git clone https://github.com/ds9-code/organoid-agent.git
cd organoid-agent
```

2. Env.

```
conda create -n organoid-agent python=3.11 -y
conda activate organoid-agent
pip install -r requirements.txt
```

3. Data subset (~80 MB).

```
python download_data.py
```

4. Sanity-check.

```
python explore_hnoca.py
```

5. Regenerate plots.

```
make plots
```

6. (Optional) Run the agent.

```
cp .env.example .env          # fill in OPENAI_API_KEY
make demo
make agent
```

To clone Medea as a reference:

```
git clone https://github.com/mims-harvard/medea.git
```

Appendix D — Open questions / things to bring up with the lab

1. **Which foundation model first?** Geneformer is easiest (rank-based, HF-hosted, one-line load); scGPT and UCE need more setup. I'd start with Geneformer for v1.
2. **Are T1 numbers real?** Only 54 train pairs $\times$ 500 HVGs at `group_key=protocol`; eval set is tiny. Need bootstrap CIs.
3. **Define “novel cell” operationally.** Options: (a) latent-space outlier vs. HNOCA percentile, (b) trajectory deviation under matched protocol. Need to pick before Paola data arrives.
4. **Cell-type label space is messy.** 28 fine labels, many near-synonymous; 14% labelled `unknown`. Use `annot_level_2` (coarse) for T2 main result, fine for ablation.
5. **CellFlow vs HNOCA structure mismatch.** CellFlow was trained on perturbation pairs (control $\rightarrow$ treated). HNOCA has developmental trajectories (young $\rightarrow$ old, same protocol). Will CellFlow's OT formulation transfer, or do we need a simpler regressor?
6. **What does Paola actually have?** Time-course vs. snapshot organoids? T1 / T5 need time-course; T2 / T3 / T6 work on snapshots.
7. **For T4:** is there a planned perturbation arm in Paola's dataset, or are we always operating on the protocol-as-perturbation framing?

Appendix E — misc surprises from real data

- **14% of cells in Dorsal-telencephalon subset are labelled “unknown”** in `cell_type` — second most common class. Not noise, a real gap in cellxgene's harmonisation. Flag when reporting T2.
- **PC1 Spearman 0.25** means PC1 is *not* primarily a time axis in this subset — protocol/batch dominates. A protocol-conditional PC1 (Spearman within protocol, then averaged) is a fairer baseline.
- **The /obs streaming trick.** `obs` is small enough to read every categorical column into memory in <30s of HTTPS traffic; that's the unlock that makes the 80 MB subset possible.
- **PopMean wins MSE on T1** because between-protocol variance dominates within-protocol trajectory variance — tells you the model needs to condition on protocol identity, not just on `x_t`.
- **HuggingFace is overkill for HNOCA:** Zenodo is public, no `HF_TOKEN` needed. `.env.example` keeps OpenAI-related vars only.